Pega CSA + Constellation

Interview & Certification Preparation Guide

Prepared for a candidate with 6–24 months hands-on experience • ~2 week runway • Reflects the current PEGACPSA exam and Infinity '25 Constellation path

This guide has two jobs: get you exam-ready and get you interview-ready. The exam rewards accurate recall and scenario recognition; the interview rewards your ability to explain and defend a design out loud. Every section below is written to support both.

1. What you are actually being tested on

The CSA exam (PEGACPSA): 60 questions, 90 minutes, 65% to pass (roughly 39 correct). It is delivered through Pearson VUE, either at a test center or online with webcam proctoring. Question formats are standard multiple choice, scenario-based multiple choice, and drag-and-drop.

Domain weighting. Allocate your study time in proportion to these weights — Case Management alone is a third of the exam.

Domain	Weight	Focus
Case Management	33%	Lifecycle, stages, processes, steps, SLAs, parent/child cases
Data & Integration	23%	Data model, data pages, connectors, integration patterns
User Experience	12%	Constellation views, portals, navigation
Application Development	12%	Studios, rulesets, class hierarchy, rule resolution, app layers
Security	5%	Access groups, roles, ARO, RBAC/ABAC
DevOps	5%	Branches, merge, deployment pipeline
Reporting	5%	Report definitions, list vs summary reports
Mobility	5%	Mobile channels, offline capability

> **Interview vs exam.** The exam is recognition; the interview is articulation. At the CSA level, interviewers commonly ask you to walk through a case lifecycle you built, explain how Constellation changes what you can and cannot do, justify a data-transform-vs-activity choice, and reason about where a rule belongs in the class hierarchy. Prepare to *talk*, not just to recognize.

2. Two-week study plan

Week 1 — rebuild the foundations

- **Day 1–2 — Case management.** Case type, lifecycle, stages, processes, steps, statuses. Draw a full lifecycle from memory. Cover parent/child cases, spinoff, and case-wide vs stage-wide optional actions.
- **Day 3 — Data model & data pages.** Data classes, property modes, data page scope (node/thread/requestor), read-only vs savable, reload strategies.
- **Day 4 — Integration.** Connectors (REST/SOAP), integration classes, data pages as the integration layer, error handling.
- **Day 5 — Application development.** App Studio vs Dev Studio, rulesets & versions, class hierarchy and rule resolution, the built-on application layers.
- **Day 6 — Decisioning & automation.** Decision tables/trees, when rules, data transforms vs activities, declare expressions.
- **Day 7 — Review + first full timed mock.** 60 questions in 90 minutes.

Week 2 — Constellation, weak spots, articulation

- **Day 8–9 — Constellation deep dive** (Section 4). Treat it as its own domain — this is where mid-level candidates lose points.
- **Day 10 — UX, security, reporting, DevOps, mobility.** Breadth over depth on the smaller domains.
- **Day 11 — Attack your weakest domain** from the Day 7 mock.
- **Day 12 — Mock interview out loud** using Sections 5 and 6. Record yourself.
- **Day 13 — Second full timed mock** + review every wrong answer until you know *why*.
- **Day 14 — Light review only.** Skim your notes, rest. No new material.

3. Core concepts, explained

Case Management (33% — master this)

A **case** represents a unit of work that travels from creation to resolution. A **case type** is the reusable definition of that work — the blueprint from which case instances are created.

The **case lifecycle** models how work progresses, and is built from three nested levels:

- **Stages** — the major phases (e.g., Submission → Review → Approval → Fulfillment). Stages are usually *sequential*; *alternate* stages branch off the normal path, and *resolution* stages end the case.
- **Processes** — the flows that run within a stage.
- **Steps** — the atomic units inside a process. A step is either an *assignment* (routed to a human), an *automation* (the system performs it), or a *subprocess*.

Distinctions worth memorizing: **case-wide optional actions** are available at any point in the case, while **stage-wide optional actions** are available only within that stage. Case **status** (stored in `pyStatusWork`) reflects progress — New, Open, Pending, Resolved-Completed, Resolved-Rejected, and so on. A parent case can **spin off** child cases, and you configure whether the parent waits for children to resolve. A **Service Level Agreement (SLA)** defines goal, deadline, and passed-deadline intervals with escalation actions, and can attach at the assignment or case level.

Data & Integration (23%)

Data model. Data objects and fields authored in App Studio map to classes and properties in Dev Studio. Know the property modes: single value, value-list, value-group, page, page-list, and page-group.

Data pages are the central mechanism for retrieving and holding data:

- **Scope** — *Node* (shared by all users on a node; good for reference data), *Thread* (one per case/session thread), *Requestor* (one per user session).
- **Structure** — page (one record) or list (many).
- **Mode** — read-only (loaded automatically) or *savable* (explicitly written back to a system of record).
- **Reload strategy** — reload once per interaction, do not reload when a condition holds, or reload if older than a set age.

Integration. Connectors call outbound to external systems (REST, SOAP, etc.); services expose Pega to inbound callers. The recommended pattern is to front an external source with a data page, so the rest of the application depends on the data page rather than the connector — decoupling the app from the integration.

Application Development (12%)

- **Studios.** App Studio for business users / citizen developers authoring cases and views visually; Dev Studio for architects doing lower-level rule work; plus Prediction Studio and Admin Studio.
- **Rulesets and versions** package and version rules; the ruleset list controls which rules are visible and resolvable.
- **Class hierarchy & rule resolution.** Pega resolves a rule by walking up the class hierarchy while honoring ruleset order, version, availability, and circumstancing. You specialize a rule by class or by circumstance.
- **Application layers.** The built-on stack — *organization* layer (shared enterprise assets), *framework* layer (reusable across implementations), *implementation* layer (the concrete application). This structure is what makes reuse possible.

The smaller domains (5–12%)

- **User Experience.** In Constellation, UI is authored as *Views* in App Studio rather than sections/harnesses. Layout is prescribed by templates. (Full detail in Section 4.)
- **Security.** Role-based access via *access groups* → *roles* → *Access of Role to Object (ARO)* rules for class-level privileges, with *Access When* rules for conditional access. ABAC adds policy-driven field/instance restrictions.
- **DevOps.** Development branches, merge, and the deployment pipeline (Deployment Manager / CI-CD).
- **Reporting.** Report Definition rules; *list reports* (row detail) vs *summary reports* (aggregation).
- **Mobility.** Mobile channels, the Pega Mobile client, and offline-capable apps at a conceptual level.

4. Constellation deep dive

This is the section that separates a confident CSA candidate from an average one. Learn it well enough to explain without notes.

The core shift: server-rendered to client-rendered

Traditional (Theme-Cosmos / section-based) Pega: the Infinity server did double duty — it ran business logic *and* generated all the UI markup (harnesses, sections, HTML), returning fully rendered pages to the browser. That meant heavy server load, chatty round-trips, and UI tightly coupled to data.

Constellation: the server stops rendering UI. It sends **data plus UI metadata** as JSON, and a React-based design system assembles and renders the interface *on the client*. This is a fundamental architectural change, not a visual refresh.

MVC mapping (a favorite interview question)

- **Model** — the Pega Infinity Platform: business logic, workflow, data.
- **View** — the Constellation design system (React), running in the browser, rendering the actual UI.
- **Controller** — the Constellation orchestration layer, also client-side; it captures user actions, holds client-side state, and translates actions into API calls.
- Model and Controller communicate through the **DX API v2**.

The three components of Constellation

1. **Constellation design system** — a React library of pre-built UI components and *patterns* (proven component combinations). Constellation apps are **Single Page Applications (SPAs)**: one HTML page that rewrites itself dynamically instead of loading new pages.
2. **Constellation architecture** — the stateless, client-orchestrated runtime. UI rendering, client-side state, and orchestration all happen on the device.
3. **Constellation DX API v2** — a set of model-driven REST endpoints returning data and view metadata.

DX API v2 — what to remember

- It is **model-driven** and **view-based** (authored as Views in App Studio), replacing the harness/section coupling of the traditional DX API.
- It cleanly **separates data from presentation** in the JSON and reduces chattiness — fewer round trips and a much smaller payload on first load.
- The **same endpoints** power both Pega's out-of-the-box UI and any custom front-end (React, Angular, Vue, vanilla JS) — the "build once, render anywhere" story.
- It follows **HATEOAS** — responses include references to the next available actions (e.g., endpoints to follow a case or change a stage).
- Notable headers/params: `x-origin-channel` (web vs mobile, usable for circumstancing); `eTag` + `if-match` for *optimistic locking*; `viewType` (how much metadata is returned) and `pageName`. `pyDetails` is the default top-level page.

Stateless architecture

Constellation moves toward **stateless** operation: the requestor session and the Clipboard are released after each DX API call, rather than a long-lived server session holding state. State lives on the client. This is why Constellation scales better, and it is the key contrast to draw against the traditional stateful, Clipboard-heavy model.

Configuration over customization

Constellation deliberately favors **configuration over customization**. It ships *prescribed UI* — predefined templates, navigation, and patterns distilled from Pega's decades of enterprise UX. As a CSA you *configure* views and select templates rather than hand-building pixel layouts. When genuine custom UI is required, you build a **Constellation DX Component** in React, using the `PCore` and `PConnect` APIs, which plugs directly into View authoring in App Studio.

Static content delivery (CDN)

Constellation's framework JavaScript (Cosmos-React plus Pega-generated code) is served from a global **Pega CDN**, while client-authored assets (images, custom components, localizations) come from the application-specific **App-Static** service. Knowing this split signals that you understand the deployment picture, not just the coding one.

Constellation vs traditional — comparison

Aspect	Traditional (Cosmos/sections)	Constellation

UI rendering	Server-side	Client-side (React)
State	Stateful (Clipboard, long session)	Stateless (released per call)
Authoring	Harnesses & sections (Dev Studio)	Views (App Studio)
Coupling	UI tightly coupled to data	Data and presentation separated
Philosophy	Customization-friendly	Configuration over customization
API	Traditional DX API v1	DX API v2 (model-driven, HATEOAS)
Front-end reuse	Limited	Any framework via DX API

Gotchas interviewers probe: not everything migrates one-to-one — some traditional customizations and rules are not Constellation-compatible, so a readiness/compatibility check matters. DX-API best practice is that sections should auto-generate and visibility expressions should reference *when* rules. And you give up free-form pixel control — that is the deliberate trade for consistency, stability, and easy upgrades.

5. Scenario-based questions with answers

These mirror the exam's scenario style and the interview's "what would you do" probes. Read the scenario, form your own answer, then compare.

Case Management

S1. An insurance claim must go through Intake, then either a Fast-Track path for claims under $500 or a Detailed Review path for larger claims, before reaching Payment. Claims flagged as fraudulent must be able to jump out of the normal flow for investigation at any point. How do you model this?

Model answer: Model Intake, then a decision that routes to one of two processes — Fast-Track or Detailed Review — based on claim amount (a decision table or when rule on the amount). Both converge on a Payment stage. For the fraud path, use an **alternate stage** "Investigation" reachable via a **case-wide optional action**, so it can be triggered from anywhere rather than only within one stage. Routing between Fast-Track and Detailed Review is a branching decision inside the lifecycle, not two separate case types — it is the same work with different handling.

Case Management

S2. A loan application (parent) needs three independent verifications — credit, income, and identity — done in parallel by different teams. The loan cannot proceed to Approval until all three finish. How do you build it?

Model answer: Spin off three **child cases**, one per verification, from the parent loan case. Configure the parent to **wait for all children to resolve** before advancing to Approval. Running them as children (rather than sequential steps) gives true parallel work, independent routing to each team, and independent SLAs per verification, while the parent gates progression on their completion.

Data & Integration

S3. Your app shows a dropdown of country codes on many screens. The list rarely changes and comes from an external reference service. Users complain the screens are slow. What data page design fixes this?

Model answer: Use a **read-only, node-scoped data page** sourced from the reference service, with a **reload-if-older-than** strategy (e.g., refresh daily). Node scope means the list is loaded once per node and shared across all users, eliminating repeated per-user calls to the external service. Because the data is stable reference data, an age-based reload keeps it fresh without hammering the source — directly addressing the slowness.

Data & Integration

S4. A screen lets a user edit customer contact details and save them back to an external CRM. Which data page mode do you use, and why not a plain read-only page?

Model answer: Use a **savable data page**. A read-only data page only *loads* data and cannot write changes back to the source. A savable data page is designed to persist edited data to the system of record (the CRM) when explicitly saved, which is exactly what an editable, write-back screen requires. You would also typically front the CRM through this data page so the UI is decoupled from the connector.

App Development

S5. Two applications built on the same framework both need a "calculate shipping cost" rule, but one application in a specific region needs slightly different logic. Where do you place the rules and how do you handle the exception?

Model answer: Put the standard "calculate shipping cost" rule in the **framework layer** so both applications inherit it. For the regional exception, **specialize** the rule — either by placing an overriding version in that application's *implementation* ruleset, or by **circumstancing** the rule on the region property. Rule resolution then serves the specialized version for that region and the framework default everywhere else, giving reuse plus a targeted override without duplicating the whole rule.

App Development

S6. A developer reports that their updated activity "isn't being picked up" even though they saved it. What likely rule-resolution factors would you check?

Model answer: Check the **ruleset list and order** for the user's access group — a higher-priority ruleset may hold an older version that wins. Check the **ruleset version** they saved into versus what resolves. Check **availability** (is the new rule set to Available, or is an old one blocking/withdrawn?). Check **class specialization** — a more specific class may carry an overriding copy. And confirm they are testing in the right **access group / application context**. Rule resolution walks class hierarchy plus ruleset order/version/availability, so the culprit is almost always one of those.

Constellation

S7. Leadership wants to migrate a heavily customized Theme-Cosmos application to Constellation and expects a like-for-like UI with all existing custom sections preserved. How do you set expectations as the CSA?

Model answer: Explain that Constellation favors **configuration over customization** and uses *prescribed* templates and patterns, so a pixel-for-pixel port of bespoke sections is not the goal — and not everything migrates one-to-one. I would run a **Constellation readiness/compatibility assessment** to identify incompatible customizations, re-author screens as **Views in App Studio**, and where a genuinely custom element is unavoidable, build a **Constellation DX Component** in React. The payoff to communicate: better performance, stability, and dramatically easier upgrades, in exchange for giving up free-form layout control.

Constellation

S8. During a Constellation project, another architect says "the server renders the screen and sends it to the browser, same as always." Correct them and explain the runtime flow.

Model answer: That describes the traditional model, not Constellation. In Constellation the server no longer renders UI markup. Instead: the client requests data via **DX API v2**; the Infinity server (the *Model*) runs business logic and returns **data plus UI metadata as JSON**; the *Controller* (client-side orchestration layer) receives it and instructs the React *View* (the design system) to render the screen in the browser. User actions are captured client-side and translated back into DX API calls. The server never emits HTML for the UI, and the session/Clipboard is released after each call — it is client-rendered and stateless.

Constellation

S9. A front-end team wants to build a custom React portal but still use all of Pega's case logic. Is that possible in Constellation, and how?

Model answer: Yes — this is a core Constellation strength. Because the **DX API v2** exposes case data and UI metadata through model-driven REST endpoints, the *same* endpoints that power Pega's out-of-the-box UI can power a custom front-end in React, Angular, Vue, or vanilla JS. The team consumes the DX API to create, view, and advance cases — and HATEOAS responses tell them the available next actions — while all business logic and workflow stay on the Pega server. They get full control of presentation without re-implementing case logic.

Security

S10. Managers should see and approve expense cases; regular employees should only submit and view their own. How do you set this up?

Model answer: Define two **roles** mapped to two **access groups** (Manager and Employee). Use **Access of Role to Object (ARO)** rules to grant the Manager role higher privileges (open, modify, approve) on the expense case class, and restrict the Employee role to create and read. Use an **Access When** rule so employees can only view cases where they are the submitter. For finer, data-driven restrictions you could layer **ABAC** policies, but role + ARO + Access When covers this scenario cleanly.

Data / Decisioning

S11. A field on a form should always equal quantity multiplied by unit price and update instantly as the user types — no submit required. Data transform, activity, or something else?

Model answer: Use a **declarative rule** — specifically a *declare expression*. Declarative rules recalculate automatically whenever an input changes, which is exactly the "update instantly, no submit" behavior required. A data transform or activity would run only when explicitly invoked (e.g., on submit), so they are the wrong tools here. Reserve data transforms for setting/mapping values procedurally and activities for multi-step processing that declarative rules cannot express.

Reporting

S12. Management wants a dashboard showing the count of open cases grouped by region and status. List report or summary report?

Model answer: A **summary report**. Summary reports aggregate — they group rows and compute counts/totals — which matches "count of open cases grouped by region and status." A list report shows individual case rows and would not perform the grouping/aggregation the dashboard needs. Build it as a Report Definition with region and status as grouping columns and a count aggregate, filtered to open cases.

6. Rapid-fire interview questions

Practice answering each in 30–60 seconds, out loud.

1. Difference between a stage, a process, and a step.
2. Alternate stage vs optional action — when each?
3. How do parent and child cases coordinate resolution?
4. Explain SLA goal/deadline/escalation and where you attach one.
5. Node vs thread vs requestor data page scope, with a use case for each.
6. Read-only vs savable data page.
7. Why front an external system with a data page?
8. App Studio vs Dev Studio — who works where and why?
9. Explain rule resolution end to end.
10. Describe the built-on application layers and why they enable reuse.
11. Data transform vs activity vs declarative rule.
12. What problem does Constellation solve versus the older UI?
13. Map Constellation onto MVC.
14. What does "stateless" mean in Constellation, and how does it differ from the Clipboard model?
15. How does DX API v2 differ from the traditional DX API?
16. What does "configuration over customization" mean in practice for a CSA?
17. When can't you use out-of-the-box configuration, and what do you do then?
18. Where does Constellation's JavaScript come from at runtime?

> **Answer shape that scores well:** define the concept, state the relationship or trade-off, then close with *why it matters*. Interviewers reward the "why" far more than the definition alone.

7. Final-stretch resources

- **Pega Academy** — the official CSA mission, now the Infinity '25 path including Constellation and the GenAI tooling (Blueprint, AI Coach, Knowledge Buddy). Treat Academy as your source of truth.
- **Pega Community blogs** — "Characteristics of Constellation Architecture" and "Constellation Building Blocks" are accurate, well-written primers.
- **Two full timed mock exams** minimum. Use free question banks to find gaps, not to memorize answers — treat exact wording skeptically.
- **Build one small case type end to end** in a personal Pega environment if you have access. Nothing cements case lifecycle and data pages like doing it.

> **Bottom line:** with two weeks and hands-on experience you are well within reach. Put the most time into Case Management (a third of the exam) and into being able to *speak* fluently about Constellation — that combination is what convinces an interviewer.